

Формат sit-файла

Информация о формате sit-файлов не предназначена для использования при решении учебных задач по программированию Робота, она носит чисто технический характер. Решение задач должно основываться исключительно на публичном API Робота.

Файлы с расширением `.sit` предназначены для хранения обстановки на поле Робота. Формат является текстовым и человекочитаемым.

В sit-файле сохраняются:

- размеры видимой части поля;
- признак наличия внешней рамки;
- положение Робота;
- температура клеток;
- положение маркеров;
- положение внутренних перегородок;
- вспомогательный коэффициент `coefficient`.

Формат `.sit` представляет описание обстановки, а не сериализованное внутреннее представление объекта Julia.

Общая структура файла

Файл состоит из секций, расположенных в фиксированном порядке:

```
frame_size:
...

coefficient:
...

is_framed:
...

robot_position:
...

temperature_map:
...

markers_map:
...

borders_map:
...
```

В текущей реализации при чтении файла названия секций непосредственно не проверяются. Строка заголовка пропускается, после чего считываются данные соответствующей секции.

Поэтому порядок секций является обязательной частью формата.

Размеры поля

Секция

```
frame_size:  
<num_rows> <num_cols>
```

задаёт размеры видимой части поля.

`num_rows` — число строк, `num_cols` — число столбцов.

Например,

```
frame_size:  
10 12
```

задаёт поле размером 10 строк на 12 столбцов.

В программе размеры хранятся в виде пары

```
(num_rows, num_cols)
```

и определяют размеры матриц `temperature_map` и `borders_map`.

Коэффициент `coefficient`

Секция

```
coefficient:  
<value>
```

содержит вещественное число типа `Float64`.

Например:

```
coefficient:  
1.0
```

При создании новой обстановки его значение вычисляется по формуле

```
12 / max(frame_size...)
```

Таким образом, например, для поля размером 11 × 12

```
coefficient = 1.0
```

В текущей реализации Makie этот параметр сохраняется в структуре данных и в sit-файле, хотя непосредственно при отрисовке обстановки не используется.

Внешняя рамка

Секция

```
is_framed:  
<true | false>
```

задаёт наличие внешней рамки поля.

Например,

```
is_framed:  
true
```

означает, что видимая часть поля окружена внешней рамкой.

Внешняя рамка не входит в `borders_map`. Она представляется отдельно единственным значением `is_framed`.

Положение Робота

Секция

```
robot_position:  
<row> <column>
```

задаёт положение Робота.

Первое число является номером строки, второе — номером столбца.

Например,

```
robot_position:  
3 5
```

означает положение

```
(3, 5)
```

Нумерация соответствует индексированию матрицы: первая координата изменяется при движении по вертикали, вторая — по горизонтали.

Направления движения определены следующим образом:

```
Nord — уменьшение номера строки;  
Sud  — увеличение номера строки;  
West — уменьшение номера столбца;  
Ost  — увеличение номера столбца.
```

Если поле не имеет внешней рамки, Робот может выйти за пределы его видимой части. Поэтому координаты Робота не обязаны находиться внутри диапазонов `1:num_rows` и `1:num_cols` и могут, в частности, быть неположительными.

Температура клеток

Секция

```
temperature_map:  
<t1> <t2> ... <tN>
```

содержит целочисленные значения температуры всех клеток видимой части поля.

Число значений равно числу клеток:

```
N = num_rows * num_cols
```

В программе температуры хранятся в объекте типа

```
Matrix{Int}
```

и при загрузке каждое значение читается как `Int`.

Значения записываются в линейном порядке обхода матрицы Julia, то есть **по столбцам**.

Например, матрица

```
1 3  
2 4
```

будет записана как

```
temperature_map:  
1 2 3 4
```

При загрузке эта последовательность преобразуется обратно в матрицу размера `frame_size`.

Для позиций Робота за пределами видимой части поля функция `temperature` возвращает `0`; эти значения в `sit`-файле не хранятся.

Маркеры

Секция

```
markers_map:  
...
```

описывает множество клеток, содержащих маркеры.

Каждая клетка представляется парой

```
(row, column)
```

Например:

```
markers_map:  
(1, 2)(3, 2)(4, 5)
```

означает наличие маркеров в клетках

```
(1, 2)
(3, 2)
(4, 5)
```

Разделитель между соседними парами отсутствует.

Если маркеров нет, строка после заголовка остаётся пустой:

```
markers_map:
```

В программе позиции маркеров хранятся в виде

```
Set
```

Поэтому порядок перечисления маркеров не имеет смыслового значения.

Например,

```
(1, 2)(3, 5)
```

и

```
(3, 5)(1, 2)
```

описывают одно и то же множество маркеров.

Внутренние перегородки

Секция

```
borders_map:
```

описывает внутренние перегородки.

После заголовка следует ровно

```
num_rows * num_cols
```

строк — по одной строке для каждого элемента матрицы `borders_map`.

В программе эта структура имеет тип

```
Matrix{Set{HorizonSide}}
```

Каждой клетке соответствует множество направлений, в которых с этой клеткой связаны внутренние перегородки.

Клетки перечисляются в линейном порядке матриц Julia, то есть по столбцам.

Кодирование направлений

Тип `HorizonSide` определён как

```
@enum HorizonSide Nord=0 West=1 Sud=2 Ost=3
```

Поэтому в `sit`-файле используются следующие числовые обозначения:

```
0 — Nord  
1 — West  
2 — Sud  
3 — Ost
```

Если, например, строка имеет вид

```
0 3
```

то в соответствующем элементе `borders_map` находятся направления `Nord` и `Ost`.

Если множество пусто, соответствующая строка файла также является пустой.

Представление одной перегородки

Перегорodka между двумя соседними клетками является одним физическим объектом, хотя её можно описать с двух сторон.

Например, перегородка между клетками (i, j) и $(i, j+1)$ может рассматриваться как

Ost

для клетки (i, j) или как

West

для клетки $(i, j+1)$.

В `borders_map` такая перегородка хранится только в одном из этих двух представлений.

При проверке наличия перегородки программа сначала ищет соответствующее направление в множестве текущей клетки. Если его там нет, проверяется соседняя клетка и противоположное направление.

Таким образом, одна внутренняя перегородка не должна обязательно дублироваться в двух соседних элементах `borders_map`.

Внешняя рамка поля в этой секции не хранится. Она задаётся отдельно значением `is_framed`.

Пример структуры sit-файла

Условный sit-файл для небольшого поля может иметь следующий вид:

```
frame_size:
2 3
coefficient:
4.0
is_framed:
true
robot_position:
2 1
temperature_map:
10 20 30 40 50 60
markers_map:
(1, 2)(2, 3)
borders_map:
3

0

1
```


После строки `borders_map`: в данном случае должно находиться ровно шесть строк, поскольку поле содержит $2 * 3 = 6$ клеток.

Следует учитывать, что и `temperature_map`, и `borders_map` записываются в порядке линейного обхода матриц Julia — по столбцам.

Назначение формата

Формат `.sit` является предметным форматом описания обстановки на поле Робота.

Он позволяет отделить сохраняемое состояние учебной среды от деталей внутренней реализации программы.

Sit-файлы могут использоваться для:

- подготовки начальных обстановок учебных задач;
- загрузки ранее подготовленной обстановки;
- сохранения результата работы программы;
- просмотра и ручного редактирования данных в текстовом виде;
- автоматизированной проверки результатов выполнения пользовательских программ.